

Exercise 1: Configuring a Basic Spring Application

Scenario: Your company is developing a web application for managing a library. You need to use the Spring Framework to handle the backend operations.

1. **Set Up a Spring Project:**

- Created a project named as LibraryManagement Application

POM.xml :

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.library</groupId>
    <artifactId>LibraryManagement</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <java.version>21</java.version>
        <spring.boot.version>3.3.0</spring.boot.version>
    </properties>

    <dependencies>
        <!--This dependencies is for spring core -->
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context</artifactId>
            <version>6.2.7</version>
        </dependency>
        <!--This dependencies is for Spring Boot -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter</artifactId>
            <version>${spring.boot.version}</version>
```

```

</dependency>
</dependencies>

<build>
  <plugins>
    <!-- This plugin is to compile -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.12.1</version>
      <configuration>
        <release>${java.version}</release>
      </configuration>
    </plugin>

    <!-- Execution plugin-->
    <plugin>
      <groupId>org.codehaus.mojo</groupId>
      <artifactId>exec-maven-plugin</artifactId>
      <version>3.5.1</version>
      <configuration></configuration>
    </plugin>
  </plugins>
</build>
</project>

```

2. Configure the Application Context:

- Create an XML configuration file named **applicationContext.xml** in the **src/main/resources** directory.
- Define beans for **BookService** and **BookRepository** in the XML file.

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd">

```

```

    <bean id="bookRepository" class="com.library.exercise1.repository.BookRepository"/>
    <bean id="bookService" class="com.library.exercise1.service.BookService">
        <constructor-arg ref="bookRepository"/>
    </bean>
</beans>

```

3. Define Service and Repository Classes:

- Create a package **com.library.service** and add a class **BookService**.

```

package com.library.exercise1.service;

import com.library.exercise1.repository.BookRepository;
import java.util.List;
public class BookService
{
    private final BookRepository repo;

    public BookService(BookRepository repo)
    {
        this.repo=repo;
    }

    public void printAvailableBooks()
    {
        List<String> books=repo.findAll();
        books.forEach(System.out::println);
    }
}

```

- Create a package **com.library.repository** and add a class **BookRepository**.

```

package com.library.exercise1.repository;

import java.util.Arrays;
import java.util.List;
public class BookRepository
{
    public List<String> findAll()
    {
        return Arrays.asList
        (
            "1. Head First Java:",
            "2. Effective Java",
            "3. Clean Code",

```

```
"4. Design Patterns",  
"5. Java: The Complete Reference",  
"6. Java Concurrency in Practice:",  
"7. Spring in Action:",  
"8. Test-Driven: TDD and Acceptance TDD for Java Developers",  
"9. Thinking in Java ",  
"10.Java: A Beginner's Guide"  
    );  
}  
}
```

4.Run the Application:

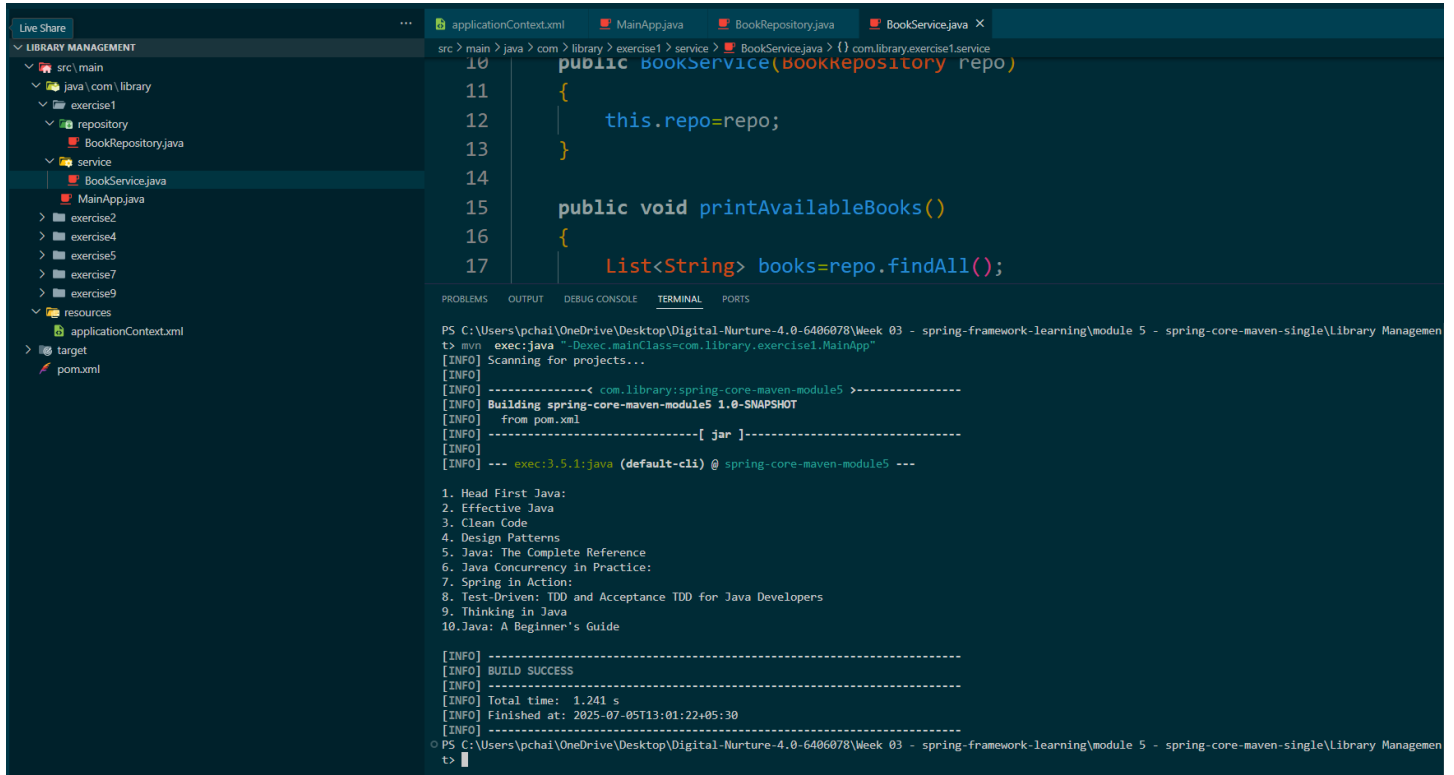
- Create a main class to load the Spring context and test the configuration.

MainApp.Java :

```
package com.library.exercise1;  
  
import com.library.exercise1.service.BookService;  
import org.springframework.context.support.ClassPathXmlApplicationContext;  
  
public class MainApp  
{  
    public static void main(String[] args)  
    {  
        try (ClassPathXmlApplicationContext ctx=new  
            ClassPathXmlApplicationContext("applicationcontext.xml"))  
        {  
            BookService service=ctx.getBean(BookService.class);  
  
            System.out.println("");  
            service.printAvailableBooks();  
            System.out.println("");  
        }  
    }  
}
```

Spring Core and Maven

OUTPUT :



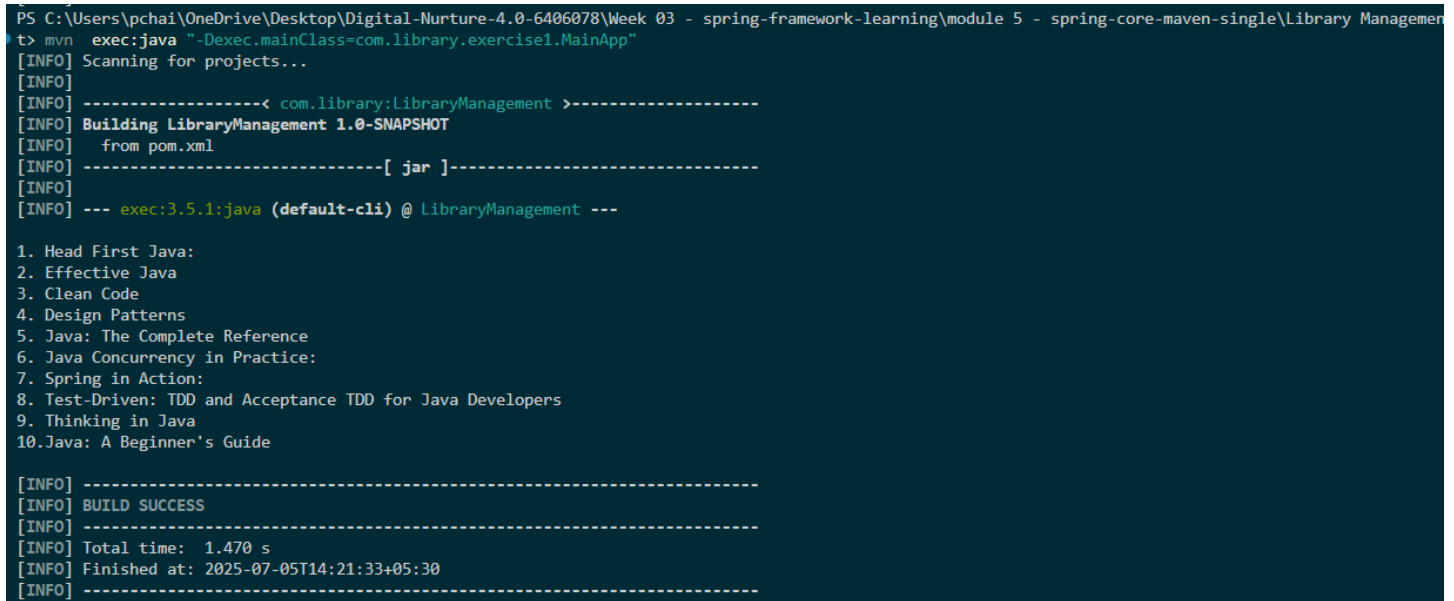
The screenshot shows an IDE with a project structure on the left and a code editor on the right. The project structure includes a 'src/main/java' directory with a 'com.library.exercise1' package containing 'BookRepository.java', 'BookService.java', and 'MainApp.java'. The code editor displays the 'BookService.java' file with the following code:

```
10 public BookService(BookRepository repo)
11 {
12     this.repo=repo;
13 }
14
15 public void printAvailableBooks()
16 {
17     List<String> books=repo.findAll();
18 }
```

The terminal window at the bottom shows the output of the command `mvn exec:java "-Dexec.mainClass=com.library.exercise1.MainApp"`. The output includes the following information:

```
PS C:\Users\pchai\OneDrive\Desktop\Digital-Nurture-4.0-6406078\Week 03 - spring-framework-learning\module 5 - spring-core-maven-single\Library Management> mvn exec:java "-Dexec.mainClass=com.library.exercise1.MainApp"
[INFO] Scanning for projects...
[INFO] -----< com.library:spring-core-maven-module5 >-----
[INFO] Building spring-core-maven-module5 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- exec:3.5.1:java (default-cli) @ spring-core-maven-module5 ---
1. Head First Java:
2. Effective Java
3. Clean Code
4. Design Patterns
5. Java: The Complete Reference
6. Java Concurrency in Practice:
7. Spring in Action:
8. Test-Driven: TDD and Acceptance TDD for Java Developers
9. Thinking in Java
10.Java: A Beginner's Guide
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.241 s
[INFO] Finished at: 2025-07-05T13:01:22+05:30
[INFO] -----
PS C:\Users\pchai\OneDrive\Desktop\Digital-Nurture-4.0-6406078\Week 03 - spring-framework-learning\module 5 - spring-core-maven-single\Library Management>
```

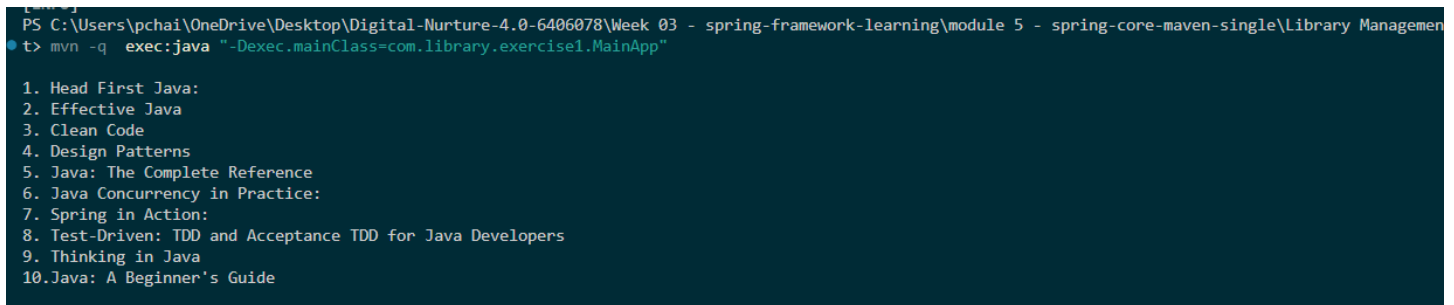
PS C:\Users\pchai\OneDrive\Desktop\Digital-Nurture-4.0-6406078\Week 03 - spring-framework-learning\module 5 - spring-core-maven-single\Library Management> mvn exec:java "-Dexec.mainClass=com.library.exercise1.MainApp"



The screenshot shows a terminal window with the following output:

```
PS C:\Users\pchai\OneDrive\Desktop\Digital-Nurture-4.0-6406078\Week 03 - spring-framework-learning\module 5 - spring-core-maven-single\Library Management> mvn exec:java "-Dexec.mainClass=com.library.exercise1.MainApp"
[INFO] Scanning for projects...
[INFO] -----< com.library:LibraryManagement >-----
[INFO] Building LibraryManagement 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- exec:3.5.1:java (default-cli) @ LibraryManagement ---
1. Head First Java:
2. Effective Java
3. Clean Code
4. Design Patterns
5. Java: The Complete Reference
6. Java Concurrency in Practice:
7. Spring in Action:
8. Test-Driven: TDD and Acceptance TDD for Java Developers
9. Thinking in Java
10.Java: A Beginner's Guide
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.470 s
[INFO] Finished at: 2025-07-05T14:21:33+05:30
[INFO] -----
```

PS C:\Users\pchai\OneDrive\Desktop\Digital-Nurture-4.0-6406078\Week 03 - spring-framework-learning\module 5 - spring-core-maven-single\Library Management> mvn -q exec:java "-Dexec.mainClass=com.library.exercise1.MainApp"



The screenshot shows a terminal window with the following output:

```
PS C:\Users\pchai\OneDrive\Desktop\Digital-Nurture-4.0-6406078\Week 03 - spring-framework-learning\module 5 - spring-core-maven-single\Library Management> mvn -q exec:java "-Dexec.mainClass=com.library.exercise1.MainApp"
1. Head First Java:
2. Effective Java
3. Clean Code
4. Design Patterns
5. Java: The Complete Reference
6. Java Concurrency in Practice:
7. Spring in Action:
8. Test-Driven: TDD and Acceptance TDD for Java Developers
9. Thinking in Java
10.Java: A Beginner's Guide
```

Exercise 2: Implementing Dependency Injection

Scenario: In the library management application, you need to manage the dependencies between the **BookService** and **BookRepository** classes using Spring's IoC and DI.

1. Modify the XML Configuration:

Update **applicationContext.xml** to wire **BookRepository** into **BookService**.

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml version="1.0" encoding="UTF-8"?>
<beans
    xmlns=http://www.springframework.org/schema/beans
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
https://www.springframework.org/schema/beans/spring-beans.xsd
">

    <bean id="emailService" class="com.library.exercise2.EmailService"/>
    <bean id="messageConsumer" class="com.library.exercise2.MessageConsumer">
        <constructor-arg ref="emailService"/>
    </bean>
</beans>
```

2. Update the BookService Class:

Ensure that **BookService** class has a setter method for **BookRepository**.

EmailService.java:

```
package com.library.exercise2;
```

```
public class EmailService implements MessageService
{
    public String getMessage()
    {
        return "You've got mail!";
    }
}
```

MessageService.java :

```
package com.library.exercise2;

public interface MessageService
{
    String getMessage();
}
```

MessageConsumer.java

```
package com.library.exercise2;

public class MessageConsumer
{
    private final MessageService service;

    public MessageConsumer(MessageService service)
    {
        this.service = service;
    }

    public void process()
    {
        System.out.println("Processing: " + service.getMessage());
    }
}
```

3. Test the Configuration:

Run the **LibraryManagementApplication** main class to verify the dependency injection.

MainApp.java :

```
package com.library.exercise2;

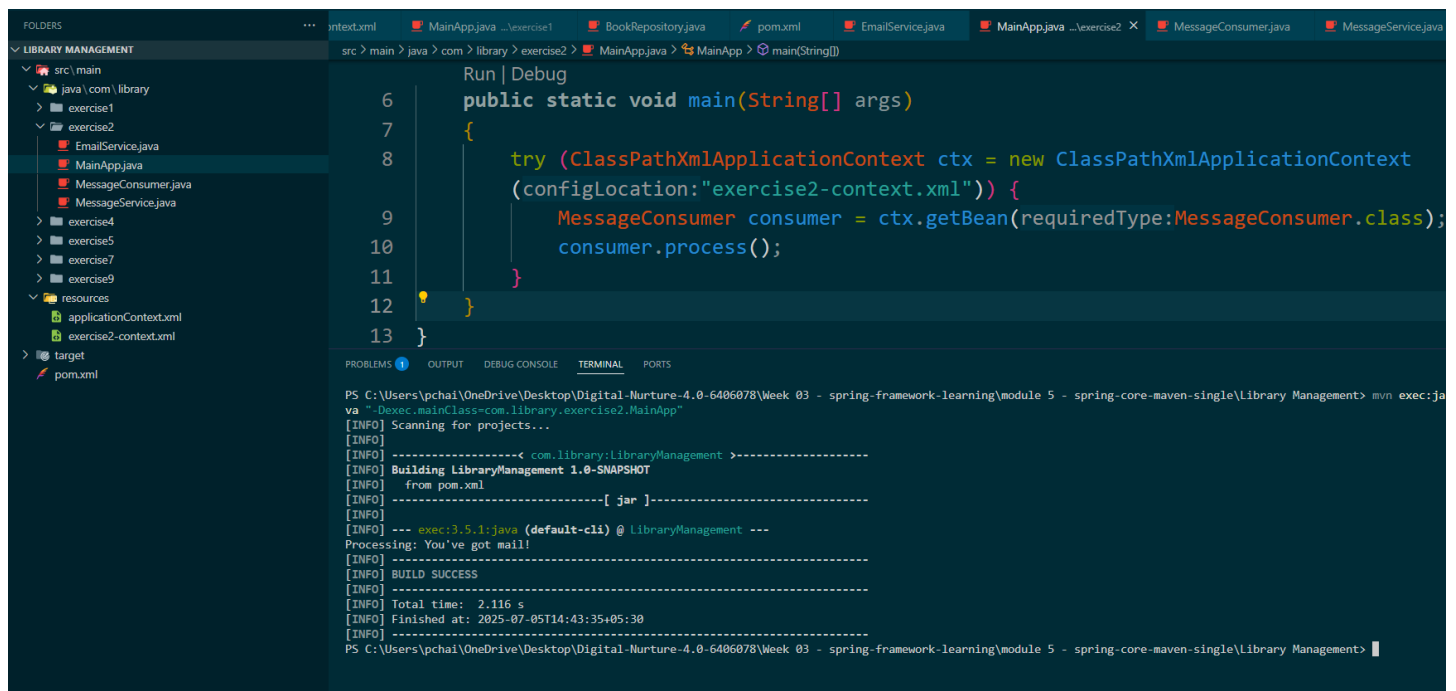
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class MainApp
{
    public static void main(String[] args)
    {
```

Spring Core and Maven

```
try (ClassPathXmlApplicationContext ctx = new ClassPathXmlApplicationContext("exercise2-context.xml"))
{
    MessageConsumer consumer = ctx.getBean(MessageConsumer.class);
    consumer.process();
}
}
```

Output :



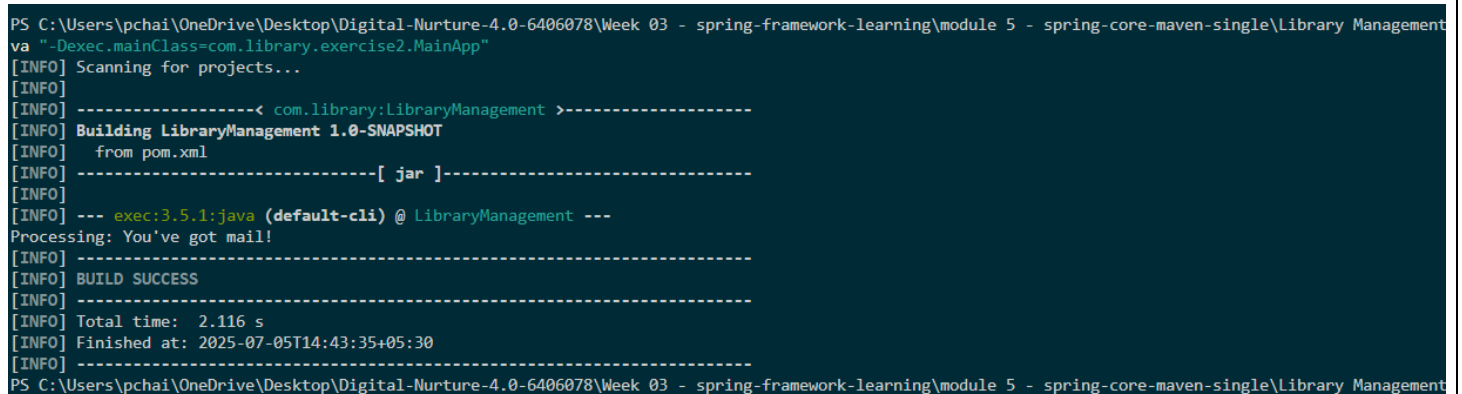
The screenshot shows an IDE with the following components:

- Left Panel (FOLDERS):** Displays the project structure under 'LIBRARY MANAGEMENT'. It includes folders for 'src/main/java/com/library' (containing 'exercise1', 'exercise2', 'exercise4', 'exercise5', 'exercise7', 'exercise9') and 'resources' (containing 'applicationContext.xml' and 'exercise2-context.xml').
- Editor:** Shows the code for 'MainApp.java'. The code is as follows:

```
6 public static void main(String[] args)
7 {
8     try (ClassPathXmlApplicationContext ctx = new ClassPathXmlApplicationContext
9         (configLocation:"exercise2-context.xml")) {
10         MessageConsumer consumer = ctx.getBean(requiredType:MessageConsumer.class);
11         consumer.process();
12     }
13 }
```
- Bottom Panel (TERMINAL):** Shows the output of the command 'mvn exec:java -Dexec.mainClass=com.library.exercise2.MainApp'. The output includes:

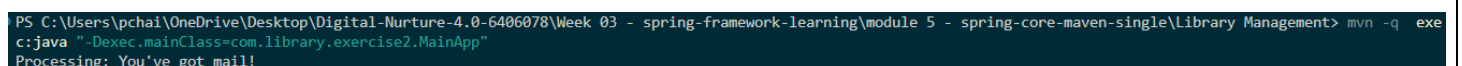
```
PS C:\Users\pchai\OneDrive\Desktop\Digital-Nurture-4.0-6406078\Week 03 - spring-framework-learning\module 5 - spring-core-maven-single\Library Management> mvn exec:java -Dexec.mainClass=com.library.exercise2.MainApp
[INFO] Scanning for projects...
[INFO] -----< com.library:LibraryManagement >-----
[INFO] Building LibraryManagement 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- exec:3.5.1:java (default-cli) @ LibraryManagement ---
Processing: You've got mail!
[INFO] BUILD SUCCESS
[INFO] Total time: 2.116 s
[INFO] Finished at: 2025-07-05T14:43:35+05:30
[INFO] -----
PS C:\Users\pchai\OneDrive\Desktop\Digital-Nurture-4.0-6406078\Week 03 - spring-framework-learning\module 5 - spring-core-maven-single\Library Management>
```

PS C:\Users\pchai\OneDrive\Desktop\Digital-Nurture-4.0-6406078\Week 03 - spring-framework-learning\module 5 - spring-core-maven-single\Library Management> mvn exec:java "-Dexec.mainClass=com.library.exercise2.MainApp"



The terminal output is identical to the one shown in the IDE screenshot above, displaying the successful execution of the application and the message 'You've got mail!'.

PS C:\Users\pchai\OneDrive\Desktop\Digital-Nurture-4.0-6406078\Week 03 - spring-framework-learning\module 5 - spring-core-maven-single\Library Management> mvn -q exec:java "-Dexec.mainClass=com.library.exercise2.MainApp"



The terminal output shows the command being executed and the message 'Processing: You've got mail!' being printed.

Exercise 4: Creating and Configuring a Maven Project

Scenario: You need to set up a new Maven project for the library management application and add Spring dependencies.

1. **Create a New Maven Project:**

- Created a new Maven project named **LibraryManagement**.

2. **Add Spring Dependencies in pom.xml:**

- Include dependencies for Spring Context, Spring AOP, and Spring WebMVC.

<parent>

 <groupId>org.springframework.boot</groupId>

 <artifactId>spring-boot-starter-parent</artifactId>

 <version>3.3.0</version>

 <relativePath/>

</parent>

<properties>

 <java.version>21</java.version>

</properties>

<dependencies>

 <!-- Core + logging -->

 <dependency>

 <groupId>org.springframework.boot</groupId>

 <artifactId>spring-boot-starter</artifactId>

 </dependency>

 <!-- AOP & MVC, matching the original brief -->

 <dependency>

 <groupId>org.springframework.boot</groupId>

 <artifactId>spring-boot-starter-aop</artifactId> <!-- adds AspectJ weaver -->

 :contentReference[oaicite:1]{index=1}

 </dependency>

 <dependency>

 <groupId>org.springframework.boot</groupId>

 <artifactId>spring-boot-starter-web</artifactId> <!-- pulls in spring-webmvc -->

 </dependency>

```
</dependencies>
```

```
<build>
```

```
<plugins>
```

```
<plugin>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-maven-plugin</artifactId>
```

```
</plugin>
```

```
</plugins>
```

```
</build>
```

3. Configure Maven Plugins:

LibraryConfig.java :

```
package com.library.exercise4;
```

```
import org.springframework.context.annotation.ComponentScan;
```

```
import org.springframework.context.annotation.Configuration;
```

```
@Configuration
```

```
@ComponentScan(basePackages="com.library.exercise4")
```

```
public class LibraryConfig {}
```

LibraryService.java :

```
package com.library.exercise4;
```

```
import java.util.List;
```

```
public interface LibraryService
```

```
{
```

```
    void addBook(String title);
```

```
    List<String> listBooks();
```

```
}
```

LibraryServiceImpl.java

```
package com.library.exercise4;
```

```
import org.springframework.stereotype.Service;
```

```
import java.util.ArrayList;
```

```
import java.util.Collections;
```

```
import java.util.List;
```

```
@Service
```

```
public class LibraryServiceImpl implements LibraryService
```

```
{
```

```
    private final List<String> catalog=new ArrayList<>();
```

```
    @Override
```

```
    public void addBook(String title)
```

```
    {
```

```
        catalog.add(title);
```

```
    }
```

```
    @Override
```

```
    public List<String> listBooks()
```

```
    {
```

```
        return Collections.unmodifiableList(catalog);
```

```
    }
```

```
}
```

MainApp.java

```
package com.library.exercise4;
```

```
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
```

```
public class MainApp
```

```
{
```

```

public static void main(String[] args)
{
    try (AnnotationConfigApplicationContext ctx=
        new AnnotationConfigApplicationContext(LibraryConfig.class)) {

        LibraryService library=ctx.getBean(LibraryService.class);

        library.addBook("Clean Code");

        library.addBook("Effective Java");

        library.addBook("Design Patterns");

        System.out.println("Current Library Collection:");

        library.listBooks().forEach(title -> System.out.println( title));

    }
}

```

Output :

The screenshot shows an IDE with a project named 'LIBRARY MANAGEMENT'. The file explorer on the left shows the project structure, including 'src/main/java/com/library/exercise4' and 'resources'. The main editor displays the code for 'MainApp.java'. The terminal at the bottom shows the output of the command 'mvn exec:java -Dexec.mainClass=com.library.exercise4.MainApp'. The output indicates that the application ran successfully, displaying the current library collection: Clean Code, Effective Java, and Design Patterns.

```

1 package com.library.exercise4;
2
3 import org.springframework.context.annotation.AnnotationConfigApplicationContext;
4
5 public class MainApp
6 {
7
8     public static void main(String[] args)

```

```

PS C:\Users\pchai\OneDrive\Desktop\Digital-Nurture-4.0-6406078\Week 03 - spring-framework-learning\module 5 - spring-core-maven-single\Library Management> mvn exec:java -Dexec.mainClass=com.library.exercise4.MainApp
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.library:LibraryManagement >-----
[INFO] Building LibraryManagement 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- exec:3.5.1:java (default-cli) @ LibraryManagement ---
Current Library Collection:
Clean Code
Effective Java
Design Patterns
[INFO] BUILD SUCCESS
[INFO] Total time: 1.146 s
[INFO] Finished at: 2025-07-05T16:58:33+05:30
[INFO]
PS C:\Users\pchai\OneDrive\Desktop\Digital-Nurture-4.0-6406078\Week 03 - spring-framework-learning\module 5 - spring-core-maven-single\Library Management>

```

PS C:\Users\pchai\OneDrive\Desktop\Digital-Nurture-4.0-6406078\Week 03 - spring-frame\module 5 - spring-core-maven-single\Library Management> mvn -q exec:java "Dexec.mainClass=com.library.exercise4.MainApp"

```

PS C:\Users\pchai\OneDrive\Desktop\Digital-Nurture-4.0-6406078\Week 03 - spring-framework-learning\module 5 - spring-core-maven-single\Library Management> mvn -q exec:java "Dexec.mainClass=com.library.exercise4.MainApp"
Current Library Collection:
Clean Code
Effective Java
Design Patterns
PS C:\Users\pchai\OneDrive\Desktop\Digital-Nurture-4.0-6406078\Week 03 - spring-framework-learning\module 5 - spring-core-maven-single\Library Management>

```

Exercise 5: Configuring the Spring IoC Container

Scenario: The library management application requires a central configuration for beans and dependencies.

1. Create Spring Configuration File:

- Create an XML configuration file named **applicationContext.xml** in the **src/main/resources** directory.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.library</groupId>
  <artifactId>LibraryManagement</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <java.version>21</java.version>
    <spring.boot.version>3.3.0</spring.boot.version>
  </properties>

  <dependencies>

    <!-- This dependencies is for spring core -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>6.2.7</version>
    </dependency>

    <!-- This dependencies is for Spring Boot -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter</artifactId>
      <version>${spring.boot.version}</version>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <!-- This plugin is to compile -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.12.1</version>
        <configuration>
          <release>${java.version}</release>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

```

    </plugin>

    <!-- Execution plugin-->
    <plugin>
        <groupId>org.codehaus.mojo</groupId>
        <artifactId>exec-maven-plugin</artifactId>
        <version>3.5.1</version>
        <configuration></configuration>
    </plugin>
</plugins>
</build>
</project>

```

- Define beans for **BookService** and **BookRepository** in the XML file.

AppConfig.Java

```
package com.library.exercise5;
```

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
```

@Configuration

```
public class AppConfig
{
    @Bean
    public GreetingService greetingService()
    {
        return new GreetingService();
    }
}
```

GreetingService.Java :

```
package com.library.exercise5;
```

```
public class GreetingService
{
    public void greet()
    {
        System.out.println("Hello from IoC container!");
    }
}
```

MainApp.Java :

```
package com.library.exercise5;
```

```
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
```

```
public class MainApp
{

```

```

public static void main(String[] args)
{
    try (AnnotationConfigApplicationContext ctx=new AnnotationConfigApplicationContext(AppConfig.class))
    {
        GreetingService gs=ctx.getBean(GreetingService.class);
        gs.greet();
    }
}
}

```

Output :

PS C:\Users\pchai\OneDrive\Desktop\Digital-Nurture-4.0-6406078\Week 03 - spring-framework-learning\module 5 - spring-core-maven-single\Library Management> mvn exec:java "-Dexec.mainClass=com.library.exercise5.MainApp"

The screenshot shows an IDE with the following components:

- Left Panel (FOLDERS):** Displays the project structure under 'LIBRARY MANAGEMENT'. It includes folders for 'src/main' (with subfolders 'java' and 'resources') and 'target'. The 'java' folder contains 'com' (with subfolders 'library' and 'exercise5'), and 'resources' contains 'applicationContext.xml' and 'exercise2-context.xml'.
- Editor:** Shows the 'AppConfig.java' file with the following code:


```

1 package com.library.exercise5;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5
6 @Configuration
7 public class AppConfig
8 {
9     @Bean
10    public GreetingService greetingService()
11    {
12        return new GreetingService();
13    }
14 }
      
```
- Terminal:** Displays the output of the command 'mvn exec:java "-Dexec.mainClass=com.library.exercise5.MainApp"'. The output includes:


```

PS C:\Users\pchai\OneDrive\Desktop\Digital-Nurture-4.0-6406078\Week 03 - spring-framework-learning\module 5 - spring-core-maven-single\Library Management> mvn exec:ja
va "-Dexec.mainClass=com.library.exercise5.MainApp"
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.library:LibraryManagement >-----
[INFO] Building LibraryManagement 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- exec:3.5.1:java (default-cli) @ LibraryManagement ---
Hello from IoC container!
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 1.358 s
[INFO] Finished at: 2025-07-05T21:02:31+05:30
[INFO]
      
```

```

PS C:\Users\pchai\OneDrive\Desktop\Digital-Nurture-4.0-6406078\Week 03 - spring-framework-learning\module 5 - spring-core-maven-single\Library Management> mvn exec:ja
va "-Dexec.mainClass=com.library.exercise5.MainApp"
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.library:LibraryManagement >-----
[INFO] Building LibraryManagement 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- exec:3.5.1:java (default-cli) @ LibraryManagement ---
Hello from IoC container!
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 1.896 s
[INFO] Finished at: 2025-07-05T21:01:32+05:30
[INFO]
PS C:\Users\pchai\OneDrive\Desktop\Digital-Nurture-4.0-6406078\Week 03 - spring-framework-learning\module 5 - spring-core-maven-single\Library Management>

```

Exercise 7: Implementing Constructor and Setter Injection

Scenario: The library management application requires both constructor and setter injection for better control over bean initialization.

Steps:

1. Configure Constructor Injection:

- Update **applicationContext.xml** to configure constructor injection for **BookService**.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="bookRepository"
        class="com.library.exercise7.BookRepository" />
  <bean id="bookService"
        class="com.library.exercise7.BookService">
    <constructor-arg ref="bookRepository" />
  </bean>
</beans>
```

2. Configure Setter Injection:

- Ensure that the **BookService** class has a setter method for **BookRepository** and configure it in **applicationContext.xml**.

BookService.java

```
package com.library.exercise7;
import java.util.List;

public class BookService
{
    private BookRepository repository;

    public BookService(BookRepository repository)
    {
```



```
        System.out.println("Constructor called");

        this.repository=repository;
    }

    public void setBookRepository(BookRepository repository)
    {
        System.out.println("Setter called");
        this.repository=repository;
    }

    public void printCatalog()
    {
        List<String> books=repository.findAll();
        books.forEach(b->System.out.println(b));
    }
}
```

BookRepository.Java

```
package com.library.exercise7;

import java.util.Arrays;
import java.util.List;

public class BookRepository
{

    public List<String> findAll()
    {
        return Arrays.asList("Clean Code", "Effective Java", "Design Patterns");
    }
}
```

3. Test the Injection:

- Run the **LibraryManagementApplication** main class to verify both constructor and setter injection.

MainApp.java :

```
package com.library.exercise7;
```

```
import org.springframework.context.support.ClassPathXmlApplicationContext;
```

```
public class MainApp
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        try (ClassPathXmlApplicationContext ctx=
```

```
            new ClassPathXmlApplicationContext("exercise7-context.xml")) {
```

```
            BookService service=ctx.getBean(BookService.class);
```

```
            System.out.println("Catalog:");
```

```
            service.printCatalog();
```

```
        }
```

```
    }
```

```
}
```

Output:

```

18      System.out.println("Setter called ");
19      this.repository=repository;
20  }
21
22  public void printCatalog()
23  {
24      List<String> books=repository.findAll();
25      books.forEach(b->System.out.println(b));
26  }
27 }
28
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\pchai\OneDrive\Desktop\Digital-Nurture-4.0-6486078\Week 03 - spring-framework-learning\module 5 - spring-core-maven-single\Library Management> mvn exec:java
va "-Dexec.mainClass=com.library.exercise7.MainApp"
[INFO] Scanning for projects...
[INFO] -----< com.library:LibraryManagement >-----
[INFO] Building LibraryManagement 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- exec:3.5.1:java (default-cli) @ LibraryManagement ---
Constructor called
Setter called
Catalog:
? Clean Code
? Effective Java
? Design Patterns
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.491 s
[INFO] Finished at: 2025-07-05T21:16:39+05:30

```

Exercise 9: Creating a Spring Boot Application

Scenario: You need to create a Spring Boot application for the library management system to simplify configuration and deployment.

BookController.java

```
package com.library.exercise9.controller;

import com.library.exercise9.entity.Book;
import com.library.exercise9.repository.BookRepository;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/books")
public class BookController
{

    private final BookRepository repo;

    public BookController(BookRepository repo)
    {
        this.repo=repo;
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Book create(@RequestBody Book book)
    {
        return repo.save(book);
    }
}
```

@GetMapping**public** List<Book> findAll()

```
{  
    return repo.findAll();  
}
```

@GetMapping("/{id}")**public** Book findById(**@PathVariable** Long id)

```
{  
    return repo.findById(id)  
        .orElseThrow(() -> new RuntimeException("Book "+id+" not found"));  
}
```

@PutMapping("/{id}")**public** Book update(**@PathVariable** Long id, **@RequestBody** Book updated)

```
{  
    return repo.findById(id).map(b -> {  
        b.setTitle(updated.getTitle());  
        b.setAuthor(updated.getAuthor());  
        return repo.save(b);  
    }).orElseThrow(() -> new RuntimeException("Book "+id+" not found"));  
}
```

@DeleteMapping("/{id}")**@ResponseStatus**(HttpStatus.NO_CONTENT)**public void** delete(**@PathVariable** Long id)

```
{  
    repo.deleteById(id);  
}  
}
```

Book.java

```
package com.library.exercise9.entity;
```

```
import jakarta.persistence.Entity;
```

```
import jakarta.persistence.GeneratedValue;
```

```
import jakarta.persistence.GenerationType;
```

```
import jakarta.persistence.Id;
```

@Entity

```
public class Book
```

```
{
```

```
    @Id
```

```
    @GeneratedValue(strategy=GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String title;
```

```
    private String author;
```

```
    public Book() { }
```

```
    public Book(String title,String author)
```

```
    {
```

```
        this.title=title;
```

```
        this.author=author;
```

```
    }
```

```
    public Long getId()
```

```
    {
```

```
        return id;
```

```
    }
```

```
    public String getTitle()
```

```
{  
    return title;  
}  
  
public String getAuthor()  
{  
    return author;  
}  
  
public void setTitle(String title)  
{  
    this.title = title;  
}  
  
public void setAuthor(String author)  
{  
    this.author=author;  
}  
}
```

BookRepository.Java

```
package com.library.exercise9.repository;  
  
import com.library.exercise9.entity.Book;  
import org.springframework.data.jpa.repository.JpaRepository;  
  
public interface BookRepository extends JpaRepository<Book, Long> {}
```

BootApplication.java

```
package com.library.exercise9;    // root package  
  
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
```

```
public class BootApp {
```

```
    public static void main(String[] args) {
```

```
        SpringApplication.run(BootApp.class, args);
```

```
    }
```

```
}
```

Output :



Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Sat Jul 05 21:56:26 IST 2025

There was an unexpected error (type=Not Found, status=404).

